实验五 数据通路与控制单元设计

一、实验目的

1 掌握 Verilog 语言和 Vivado、Logisim 开发平台的使用；

2 掌握数据通路与控制单元的设计和测试方法。

二、实验内容

1 数据通路的设计；

2 控制单元的设计。

三、实验要求

1 掌握 Vivado 或 Logisim 开发工具的使用，掌握以上电路的设计和测试方法；

2 记录设计和调试过程（Verilog 代码/电路图/表达式/真值表，Vivado 仿真结果，Logisim 验证结果等）；

3 分析 Vivado 仿真波形/Logism 验证结果，注重输入输出之间的对应关系。

四、实验过程及分析

1、主译码器：

```
module MainDec (
    input [5:0] Op,
    output MemtoReg,
    output MemWrite,
    output Branch,
    output ALUSrc,
    output RegDst,
    output RegWrite,
    output Jump,
    output [1:0] ALUOp
);
    reg [8:0] Controls;

    assign {RegWrite, RegDst, ALUSrc, Branch, MemWrite, MemtoReg, Jump, ALUOp} = Controls;

    always @(*) begin
        case (Op)
            6'b000000: Controls <= 9'b110000010; // RTYPE
            6'b100011: Controls <= 9'b101001000; // LW
            6'b101011: Controls <= 9'b001010000; // SW
```

```
            6'b000100: Controls <= 9'b000100001; // BEQ
            6'b001000: Controls <= 9'b101000000; // ADDI
            6’b001101: Controls <= 9’b101000011; // ORI
            6'b000010: Controls <= 9'b0000001xx; // J
            default:   Controls <= 9'bxxxxxxxxx; // illegal Op
        endcase
    end
endmodule
```

分析：主译码器模块根据指令的操作码（Op）生成各个控制信号，是一个纯组合逻辑电路。输入为 6 位的操作码 Op，输出包括多个控制信号：RegWrite（寄存器写使能），RegDst（目标寄存器选择），ALUSrc（ALU 第二操作数来源），Branch（分支使能），MemWrite（数据存储器写使能），MemtoReg（写回寄存器的数据来源选择），Jump（跳转信号）以及 2 位的 ALUOp（ALU 操作编码，用于进一步决定 ALU 的具体运算）。这些输出信号的意义如下：RegWrite=1 表示允许将运算结果写入寄存器；RegDst=1 表示选择指令的 rd 字段作为写寄存器地址，RegDst=0 则表示选择 rt 字段作为写寄存器地址；ALUSrc=1 表示 ALU 的第二操作数来自指令的立即数，ALUSrc=0 表示第二操作数来自寄存器；Branch=1 表示该指令是分支指令，需要根据 Zero 信号决定 PC 跳转；MemWrite=1 表示执行存储器写操作；MemtoReg=1 表示写回寄存器的数据来自数据存储器，MemtoReg=0 表示数据来自 ALU 运算结果；Jump=1 表示为跳转指令，需要无条件修改 PC；ALUOp 则用于指示 ALU 译码器应该生成何种 ALU 操作码。

主译码器内部用一个 always 组合块和 case 语句，根据不同的 Op 值赋予一个 9 位控制向量 Controls，再通过 assign 拆分到各输出端口。各指令操作码对应的控制信号组合如下：

（1）R 型指令(Op=6'b000000)：Controls 设为 9'b110000010。这意味着 RegWrite=1（需要将 ALU 计算结果写回寄存器），RegDst=1（目的寄存器号来自指令 rd 字段），ALUSrc=0（ALU 第二操作数来自寄存器 rt），Branch=0（非分支指令），MemWrite=0（不进行数据存储器写），MemtoReg=0（写回寄存器的数据来自 ALU 结果），Jump=0（非跳转），ALUOp=10（2’b10，用二进制表示，指示 ALU 译码器按 R 型指令的 Funct 字段来确定 ALU 操作）。该设置适用于所有 R 格式指令（如 ADD、SUB 等），因为 R 型指令需要 ALU 根据功能码执行相应运算，并将结果写回寄存器的 rd。

（2）LW 加载字(Op=6'b100011)：Controls 设为 9'b101001000。含义为 RegWrite=1（从数据存储器加载的数据需要写入寄存器），RegDst=0（目的寄存器使用 rt 字段，因为 LW 是 I 型指令写入 rt 寄存器），ALUSrc=1（ALU 第二操作数来自 16 位立即数，用于计算加载地址），Branch=0（不是分支），MemWrite=0（不写存储器，仅读存储器），MemtoReg=1（寄存器写回的数据来自存储器，而非 ALU 结果），Jump=0（不是跳转），ALUOp=00（2’b00，指示 ALU 执行加法操作，用于计算地址）。这样控制信号确保 LW 指令会使用 ALU 计算地址，然后从数据存储器读取数据写入寄存器 rt。

（3）SW 存储字(Op=6'b101011)：Controls 设为 9'b001010000。表示 RegWrite=0

（SW 不需要写回寄存器），RegDst=0（无意义，因为不写寄存器，但设 0），ALUSrc=1（需要用立即数计算存储地址），Branch=0（非分支），MemWrite=1（启用数据存储器写，将寄存器数据写入内存），MemtoReg=0（无意义，因不写回寄存器，但设 0），Jump=0（非跳转），ALUOp=00（执行加法计算存储地址）。这保证 SW 指令使用 ALU 计算目标内存地址并将数据写入内存，而不修改寄存器文件。

（4）BEQ 分支相等(Op=6'b000100)：Controls 设为 9'b000100001。含义为 RegWrite=0（分支指令不写寄存器），RegDst=0（无关），ALUSrc=0（ALU 需要比较寄存器操作数，不用立即数参与计算比较，相当于用两个寄存器做减法判断是否零），Branch=1（标记为分支指令），MemWrite=0（不写存储器），MemtoReg=0（不写寄存器），Jump=0（非跳转），ALUOp=01（2'b01，指示 ALU 执行减法比较操作）。当 Branch=1 时，控制单元将结合 Zero 信号产生 PCSrc，用于决定是否更新 PC 为分支目标地址。该配置使 BEQ 指令通过 ALU 减法比较两个寄存器是否相等，如果结果 Zero=1 且 Branch=1 则分支成立。

（5）ADDI 加立即数(Op=6'b001000)：Controls 设为 9'b101000000。表示 RegWrite=1（将计算结果写入寄存器），RegDst=0（目的寄存器是 rt 字段，因为 ADDI 是 I 型指令写入 rt），ALUSrc=1（ALU 第二操作数来自立即数，进行寄存器与立即数相加），Branch=0（非分支），MemWrite=0（不写存储器），MemtoReg=0（写回的数据来自 ALU 结果，而非存储器），Jump=0（非跳转），ALUOp=00（执行 ALU 加法操作）。这样 ADDI 指令的控制信号设定与 LW 类似，但无需存储器访问，ALU 直接计算寄存器与立即数之和写回。

（6）J 跳转(Op=6'b000010)：Controls 设为 9'b0000001xx。表示 RegWrite=0（跳转指令不写寄存器），RegDst=0（无关），ALUSrc=0（无关，跳转指令 ALU 结果不重要），Branch=0（不是条件分支，而是无条件跳转），MemWrite=0（不访问存储器），MemtoReg=0（不涉及写寄存器），Jump=1（跳转信号置 1），ALUOp=xx（无需 ALU 操作）。Jump=1 将引导 PC 直接跳到目标地址。

（7）ORI 按位或立即数(Op=6'b001101)：Controls 设为 9'b101000011。表示 RegWrite=1（ORI 需要将结果写入寄存器 rt），RegDst=0（写 rt），ALUSrc=1（第二操作数为立即数），Branch=0（非分支），MemWrite=0（不写存储器），MemtoReg=0（结果来自 ALU），Jump=0（非跳转），ALUOp=11（2'b11，指示 ALU 执行 OR 运算）。通过这样的控制，ORI 指令让 ALU 对寄存器和立即数执行按位或，并将结果写入目的寄存器。

（8）非法操作码(default)：Controls 设为 9'bxxxxxxxxx（未定义的 x 值）。对于未识别的 Op 码，控制单元输出不确定的 x，表示非法指令组合。在实际电路中，不期望遇到非法 Op，但在仿真时这会使输出控制信号呈未知状态，用于提示有错误或非法指令的存在。

综上，主译码器通过对 Op 的解码为每种指令类型生成正确的控制信号组合。例如，R 型指令需要 ALU 操作由 Funct 决定并写回寄存器，LW/SW 需要 ALU 计算地址及存储器读/写，分支指令需要 ALU 比较以及 Branch 信号，跳转指令需要设置 Jump 信号等。主译码器将复杂的控制信号决定逻辑封装在一个 case 结构中，使得对于每个已知指令，输出信号明确定义。

2、ALU 译码器：

```
module ALUDec (
    input [5:0] Funct,
    input [1:0] ALUOp,
    output reg  [2:0] ALUControl
);
    always @(*) begin
        case (ALUOp)
            2'b00: ALUControl <= 3'b010; // add (for lw/sw/addi)
            2'b01: ALUControl <= 3'b110; // sub (for beq)
            2'b11: ALUControl <= 3'b001; // or  (for ori)
            default: begin
                case (Funct) // R-type Instructions
                    6'b100000: ALUControl <= 3'b010; // add
                    6'b100010: ALUControl <= 3'b110; // sub
                    6'b100100: ALUControl <= 3'b000; // and
                    6'b100101: ALUControl <= 3'b001; // or
                    6'b101010: ALUControl <= 3'b111; // slt
                    default:   ALUControl <= 3'bxxx; // ???
                endcase
            end
        endcase
    end
endmodule
```

分析： ALU 译码器模块根据主译码器提供的 ALUOp 编码和指令的功能码 Funct 决定 ALU 的具体操作信号，是另一个组合逻辑单元。它的输入包括 6 位的 Funct（仅对 R 型指令有意义，对 I 型指令该字段未用）和 2 位的 ALUOp 信号。输出为 3 位的 ALUControl 信号，用于直接驱动 ALU 执行相应的运算。ALUControl 各位的编码通常对应 ALU 的操作控制，比如这里采用的编码约定：3'b010 表示加法操作，3'b110 表示减法，3'b000 表示按位与，3'b001 表示按位或，3'b111 表示比较大小取符号位，等等。ALU 译码器通过分级的 case 语句实现逻辑：首先根据 ALUOp 的值确定大类，如果 ALUOp 表示某些 I 型指令的运算，则直接给出 ALUControl；如果 ALUOp 表示 R 型运算，则进一步查看 Funct 来决定操作。具体行为如下：

- 当 ALUOp = 2'b00 时，ALU 译码器忽略 Funct，直接输出 ALUControl = 3'b010，表示执行加法操作。这种情况对应于主译码器判定的“普通算术/地址计算”类型指令，例如 LW、SW、ADDI 等，它们的 ALU 都需要进行加法运算。无论 Funct 为何值，只要 ALUOp 是 00，ALUControl 恒为 010。
- 当 ALUOp = 2'b01 时，输出 ALUControl = 3'b110，表示执行减法运算。该模式用

于分支比较指令 BEQ，主译码器对 BEQ 设置 ALUOp=01，从而 ALU 译码器输出减法控制码，让 ALU 进行两个寄存器操作数的减法以检查 Zero 标志。因此，只要检测到 ALUOp=01，ALU 就会做减法，不需查看 Funct。

- 当 ALUOp = 2'b11 时，输出 ALUControl = 3'b001，表示执行按位或操作。这种编码主要针对 ORI 这类 I 型逻辑指令。主译码器若识别到 ORI，会输出 ALUOp=11，ALU 译码器据此直接选择 OR 运算而无需 Funct 参与。
- 默认情况：ALUOp = 2’b10，主译码器用它表示 R 型指令。当 ALUOp 不匹配上述三个明确值时，ALU 译码器进入 default 分支，内部再根据 Funct 值进一步解码。这相当于处理 R 格式指令，根据功能码决定 ALU 具体操作：
    - Funct = 6'b100000（ADD）：输出 ALUControl = 3'b010（加法）。对应指令 ADD，加法结果写入寄存器。
    - Funct = 6'b100010（SUB）：输出 ALUControl = 3'b110（减法）。对应指令 SUB，执行减法计算差值。
    - Funct = 6'b100100（AND）：输出 ALUControl = 3'b000（按位与）。对应指令 AND，对两个寄存器操作数逐位与。
    - Funct = 6'b100101（OR）：输出 ALUControl = 3'b001（按位或）。对应指令 OR，对两个寄存器操作数逐位或。
    - Funct = 6'b101010（SLT）：输出 ALUControl = 3'b111（设置小于标志）。对应指令 SLT，比较大小，如果$rs<$rt 则输出 1，否则 0。
    - 默认：Funct 不属于上面列出的任何值时，输出 ALUControl = 3'bxxx。这覆盖了未实现的 R 型功能码，同时实现了 NOP 指令的要求。

通过上述逻辑，ALU 译码器有效地完成了两级解码：当主译码器给出特定 ALUOp 时，直接确定 ALU 操作；当 ALUOp 指示 R 型指令（2’b10）时，再细分 Funct 进行解码。这种分工使得主译码器无需详列所有 R 型指令的 ALU 功能，简化了主译码器的设计，也提高了可扩展性。在 Verilog 实现中，ALU 译码器使用嵌套 case 结构实现：外层判断 ALUOp，默认分支里嵌套内层判断 Funct。这种结构清晰地表达了逻辑关系，并确保当 ALUOp 与 Funct 不匹配任何已知组合时输出一个“xxx”警示。

3、控制单元：

```
module Controller (
    input [5:0] Op,
    input [5:0] Funct,
    input Zero,
    output MemtoReg,
    output MemWrite,
    output PCSrc,
    output ALUSrc,
    output RegDst,
```

```
    output RegWrite,
    output Jump,
    output [2:0] ALUControl
);
    wire [1:0] ALUOp;
    wire       Branch;

    MainDec MainDec_1 (
        .Op       (Op),
        .MemtoReg (MemtoReg),
        .MemWrite (MemWrite),
        .Branch   (Branch),
        .ALUSrc   (ALUSrc),
        .RegDst   (RegDst),
        .RegWrite (RegWrite),
        .Jump     (Jump),
        .ALUOp    (ALUOp)
    );

    ALUDec ALUDec_1 (
        .Funct      (Funct),
        .ALUOp      (ALUOp),
        .ALUControl (ALUControl)
    );

    assign PCSrc = Branch & Zero;
endmodule
```

分析：控制单元顶层模块将主译码器和 ALU 译码器组合起来，生成 CPU 数据通路所需的所有控制信号。它的输入包括 6 位操作码 Op、6 位功能码 Funct 以及来自 ALU 的 Zero 信号。输出则汇总了对数据通路各部分的控制：MemtoReg、MemWrite、PCSrc、ALUSrc、RegDst、RegWrite、Jump，以及 3 位的 ALUControl。其中 MemtoReg、MemWrite、ALUSrc、RegDst、RegWrite、Jump 等信号直接源自主译码器的输出，用于控制寄存器堆写回数据选择、数据存储器写使能、ALU 输入选择、目标寄存器选择、寄存器写使能和跳转操作等功能；ALUControl 来自 ALU 译码器的输出，用于控制 ALU 执行的具体运算；PCSrc 是控制单元内部通过 Branch 和 Zero 组合得到的一个信号，用于指示 PC 是否采用分支地址（PCSrc=1 表示发生条件跳转）。

将前两个模块进行实例化后，使用组合逻辑 assign 定义 PCSrc = Branch & Zero，将分支控制信号和 ALU 的 Zero 标志相与，实现条件分支决策。值得注意的是 Branch 信号

由 MainDec 产生，当且仅当指令为分支类型（如 BEQ）时为 1，而 Zero 信号来自 ALU 比较结果（在数据通路中由 ALU 计算）；因此 PCSrc 的结果只有在 Branch 为 1 且 ALU 比较结果为真时才为 1，从而正确地控制 PC 执行分支跳转。

4、仿真文件：

```
module sim_Controller;
    reg  [5:0] Op;
    reg  [5:0] Funct;
    reg        Zero = 1'b0;
    wire       RegWrite;
    wire       RegDst;
    wire       ALUSrc;
    wire       MemWrite;
    wire       MemtoReg;
    wire       Jump;
    wire       PCSrc;
    wire [2:0] ALUControl;

    Controller dut (
        .Op         (Op),
        .Funct      (Funct),
        .Zero       (Zero),
        .RegWrite   (RegWrite),
        .RegDst     (RegDst),
        .ALUSrc     (ALUSrc),
        .MemWrite   (MemWrite),
        .MemtoReg   (MemtoReg),
        .Jump       (Jump),
        .PCSrc      (PCSrc),
        .ALUControl (ALUControl)
    );
    initial begin
        // 测试 1: R-type 指令 - ADD
        Op = 6'b000000; Funct = 6'b100000; Zero = 1'b0; #10;

        // 测试 2: R-type 指令 - SUB
        Op = 6'b000000; Funct = 6'b100010; Zero = 1'b0; #10;
```

```
        // 测试 3: R-type 指令 - AND
        Op = 6'b000000; Funct = 6'b100100; Zero = 1'b0; #10;

        // 测试 4: R-type 指令 - OR
        Op = 6'b000000; Funct = 6'b100101; Zero = 1'b0; #10;

        // 测试 5: R-type 指令 - SLT
        Op = 6'b000000; Funct = 6'b101010; Zero = 1'b0; #10;

        // 测试 6: LW 指令
        Op = 6'b100011; Funct = 6'b000000; Zero = 1'b0; #10;

        // 测试 7: SW 指令
        Op = 6'b101011; Funct = 6'b000000; Zero = 1'b0; #10;

        // 测试 8: BEQ 指令 (Zero = 0)
        Op = 6'b000100; Funct = 6'b000000; Zero = 1'b0; #10;

        // 测试 9: BEQ 指令 (Zero = 1)
        Op = 6'b000100; Funct = 6'b000000; Zero = 1'b1; #10;

        // 测试 10: ADDI 指令
        Op = 6'b001000; Funct = 6'b000000; Zero = 1'b0; #10;

        // 测试: ORI 指令
        Op = 6'b001101; Funct = 6'b000000; Zero = 1'b0; #10;

        // 测试 11: J 指令
        Op = 6'b000010; Funct = 6'b000000; Zero = 1'b0; #10;

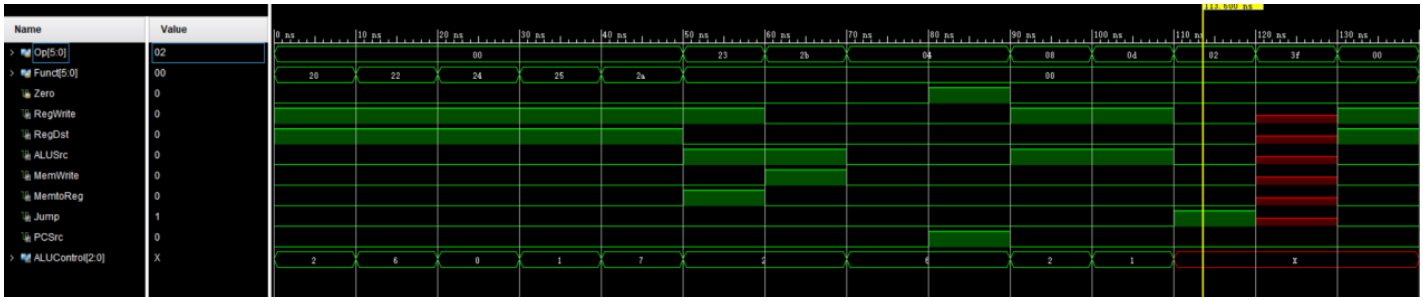
        // 测试 12: 非法操作码
        Op = 6'b111111; Funct = 6'b000000; Zero = 1'b0; #10;

        // 测试 13: Nop
        Op = 6'b000000; Funct = 6'b000000; Zero = 1'b0; #10;
        $finish;
    end
endmodule
```

仿真策略：依次施加 13 组不同指令类型的控制信号组合来验证控制电路的功能正确性。测试向量覆盖了 R 型指令、I 型指令、J 型指令、分支指令，以及非法指令和 NOP 操作等特殊情形，力求全面覆盖各种可能的控制信号输出情况。每组指令信号维持 10ns 后切换到下一组，以在时序波形中形成清晰分隔，便于观察每组指令输入下控制单元输出信号的稳定值。

通过观察仿真波形中控制单元的各输出信号，我们可以将其与预期的真值表进行比较，从而判断控制单元工作是否正确。具体来说，不同类型的指令应当激活相应的控制信号：例如 R 型指令应使能寄存器写入（RegWrite=1）、选择目标寄存器位域 rd（RegDst=1）、使用寄存器操作数（ALUSrc=0），并由功能码 Funct 决定 ALU 操作（对应不同的 ALUControl 输出）；I 型立即数算术指令（如 ADDI、ORI）应使用立即数作为 ALU 第二操作数（ALUSrc=1）且写回目标寄存器 rt（RegDst=0）；访存指令 LW 应当开启存储器读数据回写（MemtoReg=1）且使能寄存器写（RegWrite=1），SW 则应当开启存储器写使能（MemWrite=1）且不写寄存器（RegWrite=0）；分支指令 BEQ 在 opcode 解码时激活 Branch 信号，其是否真正使 PC 转移由 Zero 信号决定——我们分别测试 Zero 为 0 和 1 的情况以验证 PCSrc = Branch & Zero 的逻辑作用，其中只有当 Zero=1 时 PCSrc 才应为 1；跳转指令 J 解码后应当激活跳转信号（Jump=1）以指示 PC 采用跳转目标地址；对于非法 Op，控制单元设计为输出不定状态 x 以指示无效指令，本次也专门输入一个未定义的操作码进行测试；此外，针对 NOP 指令，我们验证控制单元是否会产生预期的“空操作”控制效果。

5、SIMULATION：



测试 1～5（R 型指令：ADD/SUB/AND/OR/SLT）：Op=000000，Funct 码分别对应不同操作。控制信号一致：RegWrite=1，RegDst=1，ALUSrc=0，MemWrite=0，MemtoReg=0，Jump=0，PCSrc=0，仅 ALUControl 根据 Funct 依次输出 010/110/000/001/111，验证 ALU 译码正确。

测试 6（LW）：Op=100011，控制信号调整为：RegWrite=1，RegDst=0，ALUSrc=1，MemtoReg=1，其余如跳转/写存储器信号为 0。ALUControl=010 执行地址加法，仿真输出正确。

测试 7（SW）：Op=101011，不写寄存器，RegWrite=0，MemWrite=1，ALUSrc=1，使用立即数地址并写存储器，ALU 同样做加法，符合 SW 控制需求。

测试 8～9（BEQ）：Op=000100，ALU 执行减法比较（ALUControl=110），当 Zero=0 时 PCSrc=0 不跳转，当 Zero=1 时 PCSrc=1 触发跳转，验证分支条件逻辑正确。

测试 10（ADDI）：Op=001000，立即数加法操作，RegWrite=1，RegDst=0，ALUSrc=1，

MemtoReg=0，ALUControl=010，波形与设计一致。

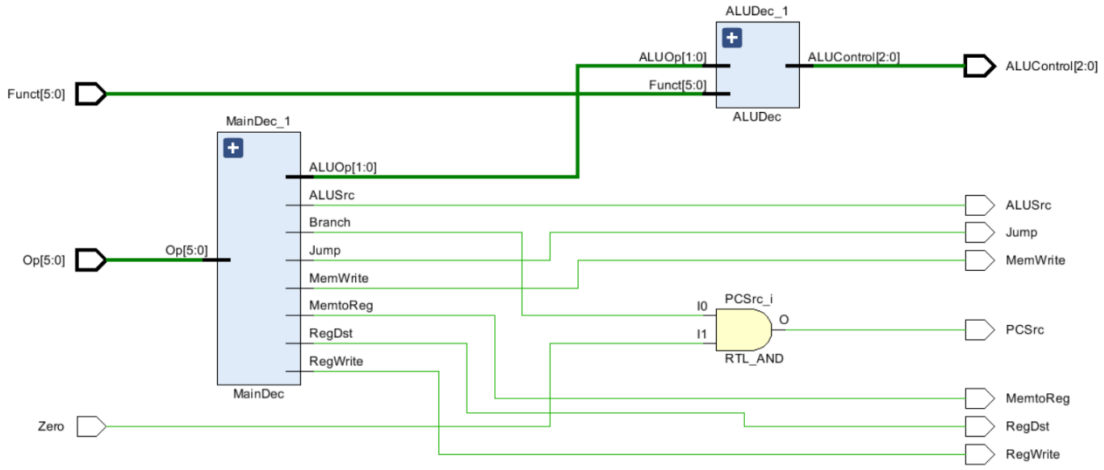
测试 11（ORI）：Op=001101，立即数或操作，控制信号与 ADDI 类似，ALUControl=001，成功执行按位或。

测试 12（J）：Op=000010，跳转指令唯一输出 Jump=1，其余信号关闭，无需使用 ALU，仿真跳转信号正确激活。

测试 13（非法指令）：Op=111111 不在译码表中，所有控制信号变为 X，验证控制器能对非法输入保持“无效”响应，防止误执行。

测试 14（NOP）：Op=000000，Funct=000000，解码为 R 型但无效操作码，ALUControl 为 X，其余控制信号仍为默认 R 型配置，符合 NOP “无实际动作”的特性。

6、RTL ANALYSIS：



分析：整个控制单元采用了分层的 RTL 结构，顶层的 Controller 模块内部包含两个子模块：MainDec 和 ALUDec，分别负责指令的大类解码和 ALU 具体操作的译码。Controller 首先将操作码 Op 传入 MainDec，生成一组主控制信号（如 RegWrite、ALUSrc、Jump、MemWrite 等）以及两位的 ALUOp 信号。随后，ALUOp 和 Funct 共同送入 ALUDec 模块，由其输出三位的 ALUControl 信号，用于驱动 ALU 执行对应的运算操作。同时，Controller 内部还使用一个与门将 MainDec 输出的 Branch 信号与外部的 Zero 信号相与，形成 PCSrc 输出，控制程序计数器是否跳转。整个控制器设计为纯组合逻辑，在 RTL 视图中，各模块通过清晰的线网连接，结构层次分明：Op 进入 MainDec，ALUOp 和 Funct 进入 ALUDec，Branch 与 Zero 决定 PCSrc，所有输出直接连接到外部数据通路。这种分层设计将指令解码任务拆分为主控和 ALU 控两个模块，既简化了单一模块的复杂度，又增强了系统的可读性和可维护性，体现了单周期 CPU 控制单元中常见的模块化设计思想。

五、调试和心得体会

1、问题与调试：

起初在编写主译码器时，容易遗漏某些指令的控制输出，例如最初忘记为 ORI 指令(Op=6'b001101)指定控制信号，导致仿真时发现当输入 ORI 指令时 ALUControl 没有得到预期的 3'b001，而是出现了未知值 xxx。这提示我注意检查指令集的完整性。通过参考指令功能，我在 MainDec 的 case 语句中增加了对 ORI 的处理，重新仿真后 ORI 的控制信号才正确显示为 OR 操作码。这个调试过程让我体会到：控制单元设计必须全面覆盖指令集中的每一种情况，即使漏掉一条指令，仿真波形中的 x 值也能迅速暴露问题。

另一个调试细节来自于 BEQ 分支逻辑。在最初的测试中，我只设置了 Branch 信号而没有正确处理 Zero 与 PCSrc 的关系，结果在波形中发现无论 Zero 为何值，PCSrc 一直未能拉高，分支似乎从不发生。意识到这一点后，我在 Controller 顶层加入了 assign PCSrc = Branch & Zero 逻辑，并在测试中专门针对 Zero=1 的情况进行验证。修正后，仿真波形清晰地出现了 PCSrc 在 Zero 为 1 时的高电平脉冲，证实分支控制正常。这个问题让我深刻认识到组合逻辑中条件信号的处理：Branch 表示“可能跳转”，但需要与运行时条件（Zero）相结合才能最终决定跳转，因此两者缺一不可。调试过程中通过针对性地改变 Zero 的测试矢量，成功验证了 PCSrc 的实现。

2、心得体会：

完成调试后，对整个控制单元设计有了更深入的理解。控制信号虽然繁多，但都有明确的作用范围：每一个信号在数据通路中控制一个多路选择器或使能信号，组合起来就指导了一条指令的执行。设计控制单元时，需要从指令的执行过程出发，推导出哪些信号应该取何值。例如，设计 MainDec 时，我反复推演每种指令的执行流程：这条指令要不要写回寄存器？要写哪个寄存器？ALU 的输入是什么？有没有存储器访问？PC 如何变化？只有明确了这些，才能正确填写控制信号真值表。这让我体会到，对数据通路的深入理解是正确设计控制单元的基础。控制单元实际上编码了处理器的“行为规则”：哪类指令激活哪类硬件单元。通过此次实验，我加深了对单周期 CPU 原理的认识，理解了为什么将控制分为主译码器和 ALU 译码器两级：这样做大大简化了问题规模，每级只需关注有限的信号和字段。例如主译码器只需根据 Op 产生大方向的控制，而 ALU 译码器专注 Funct 到 ALU 操作的映射。模块化设计和分级解码提高了电路的清晰度，也方便了调试——因为我可以分别验证 MainDec 和 ALUDec，再看它们组合后的效果。实际调试中也是如此，通过仿真观察某个信号不对，可以追溯是 Op 解码出了问题还是 Funct 解码出了问题，定位起来更加容易。

3、总结：

这次实验培养了我分析数字电路时序和逻辑的能力，让我体会到了将抽象的指令功能转化为硬件控制信号的过程。在心得方面，我认识到：严谨全面的测试对于数字设计至关重要，我们需要设计各种边界情况来验证系统的健壮性；良好的代码风格和注释也很有帮助，例如用二进制直观地列出 Controls 值并标注指令名称，方便检查错误。通过调试和思考，我对控制单元的作用有了更深的体会——它就像 CPU 的大脑，合理的设计和实现保证了 CPU 各部件协调一致地工作。此次实验不仅让我掌握了 Verilog 硬件描述和仿真技术，更重要的是深化了对计算机体系结构中控制路径设计的理解，这将为后续更复杂的 CPU 设计打下坚实基础。